

Assignment #8: 树为主

Updated 1704 GMT+8 Apr 8, 2025

2025 spring, Compiled by 李皓翔 物院

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

LC108.将有序数组转换为二叉树

dfs, <https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/>

思路:

代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(left, right):
            if left > right:
                return
            mid = (left + right + 1) // 2
            root = TreeNode(nums[mid])
            root.left = build(left, mid - 1)
            root.right = build(mid + 1, right)
            return root
```

```
return build(0, len(nums)-1)
```

代码运行截图 (至少包含有"Accepted")



M27928:遍历树

adjacency list, dfs, <http://cs101.openjudge.cn/practice/27928/>

思路：

代码：

```
def travel(root, tree):
    nodes = [root] + tree.get(root, [])
    nodes_sorted = sorted(nodes)
    for node in nodes_sorted:
        if node != root :
            travel(node, tree)
        else:
            print(node)
```

```

n = int(input())
tree = {}
root_candidate = set()
children = set()
for _ in range(n):
    parts = list(map(int, input().split()))
    node = parts[0]
    tree[node] = parts[1:]
    root_candidate.add(node)
    children.update(parts[1:])
root = (root_candidate - children).pop() if (root_candidate - children) else parts[0]
travel(root, tree)

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```

def travel(root, tree):
    nodes = [root] + tree.get(root, [])
    nodes_sorted = sorted(nodes)
    for node in nodes_sorted:
        if node != root:
            travel(node, tree)
        else:
            print(node)

n = int(input())
tree = {}
root_candidate = set()
children = set()
for _ in range(n):
    parts = list(map(int, input().split()))
    node = parts[0]
    tree[node] = parts[1:]
    root_candidate.add(node)
    children.update(parts[1:])
root = (root_candidate - children).pop() if (root_candidate - children)
travel(root, tree)

```

©2002-2022 POJ 京ICP备20010980号-1

LC129.求根节点到叶节点数字之和

dfs, <https://leetcode.cn/problems/sum-root-to-leaf-numbers/>

思路:

代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        ans = 0
        def dfs(node, s):
            if not node:
                return
            s = s * 10 + node.val
            if not node.left and not node.right:
                nonlocal ans
                ans += s
                return
            dfs(node.left, s)
            dfs(node.right, s)
        dfs(root, 0)
        return ans
```

代码运行截图 (至少包含有"Accepted")



M22158:根据二叉树前中序序列建树

tree, <http://cs101.openjudge.cn/practice/22158/>

思路:

代码:

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def buildTree(preorder, inorder):
    if not preorder or not inorder:
        return None
    root_val = preorder[0]
    root = TreeNode(root_val)
    root_pos = inorder.index(root_val)
    root.left = buildTree(preorder[1:1+root_pos], inorder[:root_pos])
    root.right = buildTree(preorder[1+root_pos:], inorder[root_pos+1:])
```

```

        return root

def postorderTraversal(root):
    if not root:
        return []
    return postorderTraversal(root.left) + postorderTraversal(root.right) +
[root.val]

tem=[]
while True:
    try:
        tem.append(input())
    except EOFError:
        break

idx = 0
while idx < len(tem):
    preorder = tem[idx].strip()
    idx += 1
    if idx >= len(tem):
        break
    inorder = tem[idx].strip()
    idx += 1
    if not preorder or not inorder:
        continue
    root = buildTree(preorder, inorder)
    postorder = postorderTraversal(root)
    print(''.join(postorder))

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def buildTree(preorder, inorder):
    if not preorder or not inorder:
        return None
    root_val = preorder[0]
    root = TreeNode(root_val)
    root_pos = inorder.index(root_val)
    root.left = buildTree(preorder[1:1+root_pos], inorder[:root_pos])
    root.right = buildTree(preorder[1+root_pos:], inorder[root_pos+1:])
    return root

def postorderTraversal(root):
    if not root:
        return []
    return postorderTraversal(root.left) + postorderTraversal(root.right)

tem=[]
while True:
    try:
        tem.append(input())
    except EOFError:
        break

idx = 0
while idx < len(tem):
    preorder = tem[idx].strip()
    idx += 1
    if idx >= len(tem):
        break
    inorder = tem[idx].strip()
    idx += 1
```

基本信息

#: 48920987
题目: 22158
提交人: 24n2400011450
内存: 3652kB
时间: 24ms
语言: Python3
提交时间: 2025-04-15 22:21:18



M24729:括号嵌套树

dfs, stack, <http://cs101.openjudge.cn/practice/24729/>

思路:

代码:

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []

def build_tree(s):
    if not s:
        return None
    root_val = s[0]
    root = TreeNode(root_val)
    if len(s) == 1:
        return root
    content = s[2:-1]
    children = []
    balance = 0
```

```

start = 0
for i, char in enumerate(content):
    if char == '(':
        balance += 1
    elif char == ')':
        balance -= 1
    elif char == ',' and balance == 0:
        children.append(content[start:i])
        start = i + 1
children.append(content[start:])
for child in children:
    root.children.append(build_tree(child))
return root

def preorder(root):
    res = []
    if root:
        res.append(root.val)
        for child in root.children:
            res.extend(preorder(child))
    return res

def postorder(root):
    res = []
    if root:
        for child in root.children:
            res.extend(postorder(child))
        res.append(root.val)
    return res

s = input().strip()
root = build_tree(s)
pre = preorder(root)
post = postorder(root)
print(''.join(pre))
print(''.join(post))

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []

def build_tree(s):
    if not s:
        return None
    root_val = s[0]
    root = TreeNode(root_val)
    if len(s) == 1:
        return root
    content = s[2:-1]
    children = []
    balance = 0
    start = 0
    for i, char in enumerate(content):
        if char == '(':
            balance += 1
        elif char == ')':
            balance -= 1
        elif char == ',' and balance == 0:
            children.append(content[start:i])
            start = i + 1
    children.append(content[start:])
    for child in children:
        root.children.append(build_tree(child))
    return root

def preorder(root):
    res = []
    if root:
        res.append(root.val)
        for child in root.children:
            res.extend(preorder(child))
    return res
```

基本信息

#: 48921210
题目: 24729
提交人: 24n2400011450
内存: 3676kB
时间: 22ms
语言: Python3
提交时间: 2025-04-15 22:44:59



LC3510.移除最小数对使数组有序II

doubly-linked list + heap, <https://leetcode.cn/problems/minimum-pair-removal-to-sort-array-ii/>

思路:

代码:

```
class Solution:
    def minimumPairRemoval(self, nums: List[int]) -> int:
        n = len(nums)
        h = []
        dec = 0
        for i, (x, y) in enumerate(pairwise(nums)):
            if x > y:
                dec += 1
            h.append((x + y, i))
        heapify(h)
        lazy = defaultdict(int)
        left = list(range(-1, n))
        right = list(range(1, n + 1))

        ans = 0
        while dec:
```

```

ans += 1

while lazy[h[0]]:
    lazy[heappop(h)] -= 1
s, i = heappop(h)

nxt = right[i]
if nums[i] > nums[nxt]:
    dec -= 1

pre = left[i]
if pre >= 0:
    if nums[pre] > nums[i]:
        dec -= 1
    if nums[pre] > s:
        dec += 1
    lazy[(nums[pre] + nums[i], pre)] += 1
    heappush(h, (nums[pre] + s, pre))

nxt2 = right[nxt]
if nxt2 < n:
    if nums[nxt] > nums[nxt2]:
        dec -= 1
    if s > nums[nxt2]:
        dec += 1
    lazy[(nums[nxt] + nums[nxt2], nxt)] += 1
    heappush(h, (s + nums[nxt2], i))

nums[i] = s
l, r = left[nxt], right[nxt]
right[l] = r
left[r] = l

return ans

```

代码运行截图 (至少包含有"Accepted")

通过 680 / 680 个通过的测试用例

Moon Velvet 提交于 2025.04.15 22:47

写题解



京东备考计划
京东备战面试



🕒 执行用时分布

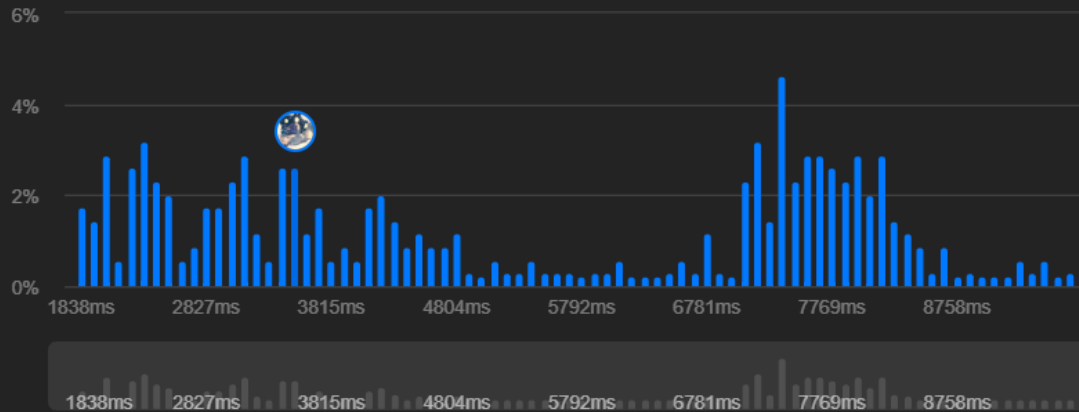


3569 ms | 击败 66.08% 🌱

🌟 复杂度分析

💾 消耗内存分布

101.25 MB | 击败 8.97%



代码 | Python3

```
class Solution:
```

2. 学习总结和收获

上周专心复习期中，作业比较仓促，最后一题读题解后才想到的